

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import pearsonr
from scipy.stats import t

pd.set_option("display.max_columns",None)
```

```
from google.colab import files
uploaded=files.upload()

Choose Files | QVI_data.csv
QVI_data.csv(text/csv) - 29019945 bytes, last modified: 5/7/2026 - 100% done
Saving QVI_data.csv to QVI_data.csv
```

```
df = pd.read_csv("QVI_data.csv")

df.head()
```

	LYLTY_CARD_NBR	DATE	STORE_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	PACK_SIZE	BRAND	
0	1000	2018-10-17	1	1	5	Natural Chip Compny SeaSalt175g	2	6.0	175	NATURAL	SINGLES
1	1002	2018-09-16	1	2	58	Red Rock Deli Chikn&Garlic Aioli 150g	1	2.7	150	RRD	SINGLES
2	1003	2019-03-07	1	3	52	Grain Waves Sour Cream&Chives 210G	1	3.6	210	GRNWVES	YOUNG
						Natural					

```
df["DATE"] = pd.to_datetime(df["DATE"])

df["YEARMONTH"] = df["DATE"].dt.strftime("%Y%m").astype(int)

df.head()
```

	LYLTY_CARD_NBR	DATE	STORE_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	PACK_SIZE	BRAND	
0	1000	2018-10-17	1	1	5	Natural Chip Compny SeaSalt175g	2	6.0	175	NATURAL	SINGLES
1	1002	2018-09-16	1	2	58	Red Rock Deli Chikn&Garlic Aioli 150g	1	2.7	150	RRD	SINGLES
2	1003	2019-03-07	1	3	52	Grain Waves Sour Cream&Chives 210G	1	3.6	210	GRNWVES	YOUNG
3	1003	2019-03-08	1	4	106	Natural ChipCo Hony Soy Chckn175g	1	3.0	175	NATURAL	YOUNG
.	1000	2018-	.	.	52	WW Original	.	4.0	100	WWSHUSPUS	

```
monthly_metrics = (

df.groupby(["STORE_NBR","YEARMONTH"])

.agg(

    Total_Sales=("TOT_SALES","sum"),

    Customers=("LYLTY_CARD_NBR","nunique"),

    Transactions=("TXN_ID","nunique"),

    Total_Qty=("PROD_QTY","sum")
```

```
)

.reset_index()

)
```

```
monthly_metrics["Txn_per_Customer"] = (

    monthly_metrics["Transactions"]/

    monthly_metrics["Customers"]

)
```

```
monthly_metrics["Chips_per_Customer"]=(

    monthly_metrics["Total_Qty"]

    /

    monthly_metrics["Customers"]

)
```

```
price=df.groupby(["STORE_NBR","YEARMONTH"]).apply(

    lambda x:x["TOT_SALES"].sum()/x["PROD_QTY"].sum()

).reset_index(name="Avg_Price")
```

```
/tmp/ipykernel_1014/1068785484.py:1: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior
price=df.groupby(["STORE_NBR","YEARMONTH"]).apply(
```

```
monthly_metrics=monthly_metrics.merge(

    price,

    on=["STORE_NBR","YEARMONTH"]

)
```

```
valid_stores=(

monthly_metrics

.groupby("STORE_NBR")

.size()

)

valid_stores=valid_stores[valid_stores==12].index

pre_trial=monthly_metrics[

(monthly_metrics["STORE_NBR"].isin(valid_stores))

&

(monthly_metrics["YEARMONTH"]<201902)

]
```

```
def calculate_correlation(metric,trial_store):

    trial=pre_trial[

        pre_trial["STORE_NBR"]==trial_store

    ][metric].values

    correlations=[]

    for store in pre_trial["STORE_NBR"].unique():

        control=pre_trial[

            pre_trial["STORE_NBR"]==store
```

```

        ][metric].values

        corr=np.corrcoef(trial,control)[0,1]

        correlations.append([trial_store,store,corr])

    return pd.DataFrame(

        correlations,

        columns=["Trial","Control","Correlation"]

    )

```

```

def calculate_magnitude(metric,trial_store):

    trial=pre_trial[

        pre_trial["STORE_NBR"]==trial_store

    ]

    result=[]

    for store in pre_trial["STORE_NBR"].unique():

        control=pre_trial[

            pre_trial["STORE_NBR"]==store

        ]

        diff=np.abs(

            trial[metric].values-control[metric].values

        )

        score=1-(

            (diff-diff.min())

            /

            (diff.max()-diff.min()+1e-10)

        )

        result.append(

            [trial_store,store,score.mean()]

        )

    return pd.DataFrame(

        result,

        columns=["Trial","Control","Magnitude"]

    )

```

```

trial_store=77

sales_corr=calculate_correlation(

    "Total_Sales",

    trial_store

)

sales_mag=calculate_magnitude(

    "Total_Sales",

    trial_store

)

```

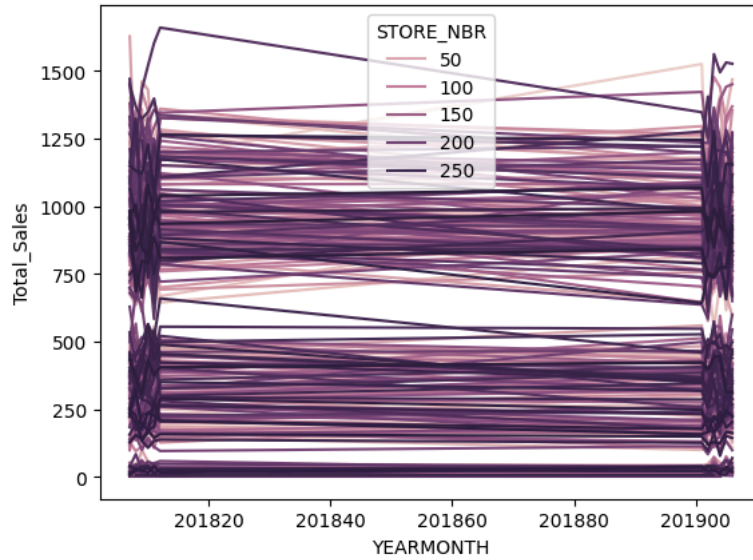
```
cust_corr=calculate_correlation(  
    "Customers",  
    trial_store  
)  
cust_mag=calculate_magnitude(  
    "Customers",  
    trial_store  
)
```

```
sales=sales_corr.merge(  
    sales_mag,  
    on=["Trial","Control"]  
)  
sales["SalesScore"]=(  
    sales["Correlation"]+  
    sales["Magnitude"]  
) / 2
```

```
sales=sales_corr.merge(  
    sales_mag,  
    on=["Trial","Control"]  
)  
sales["SalesScore"]=(  
    sales["Correlation"]+  
    sales["Magnitude"]  
) / 2
```

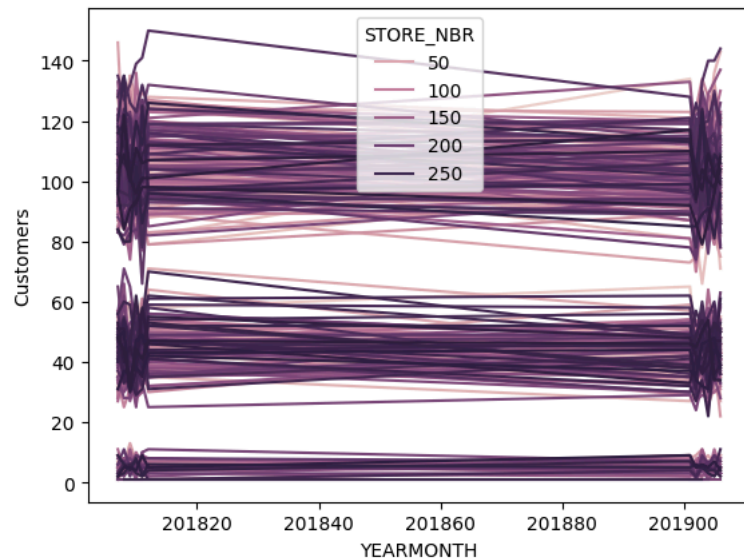
```
sns.lineplot(  
    data=monthly_metrics,  
    x="YEARMONTH",  
    y="Total_Sales",  
    hue="STORE_NBR"  
)
```

<Axes: xlabel='YEARMONTH', ylabel='Total_Sales'>



```
sns.lineplot(  
    data=monthly_metrics,  
    x="YEARMONTH",  
    y="Customers",  
    hue="STORE_NBR"  
)
```

<Axes: xlabel='YEARMONTH', ylabel='Customers'>



```
scale=(  
    pre_trial  
    .query("STORE_NBR==77")  
    ["Total_Sales"]  
    .sum()  
    /  
    pre_trial  
    .query("STORE_NBR==233")  
    ["Total_Sales"]  
    .sum()  
)
```

```
)
```

```
control_sales=monthly_metrics.query(

"STORE_NBR==233"

).copy() # Explicitly create a copy to avoid SettingWithCopyWarning

control_sales["ScaledSales"] = control_sales["Total_Sales"]*scale

# Define the 'trial' DataFrame globally for consistency and subsequent use
trial = pre_trial.query(f"STORE_NBR=={trial_store}").copy()
```

```
monthly_metrics = (
    df.groupby(["STORE_NBR", "YEARMONTH"])
        .agg(
            Total_Sales=("TOT_SALES", "sum"),
            Customers=("LYLTY_CARD_NBR", "nunique"),
            Transactions=("TXN_ID", "nunique"),
            Total_Qty=("PROD_QTY", "sum")
        )
    .reset_index()
)
```

```
print(monthly_metrics.head())
```

	STORE_NBR	YEARMONTH	Total_Sales	Customers	Transactions	Total_Qty
0	1	201807	206.9	49	52	62
1	1	201808	176.1	42	43	54
2	1	201809	278.8	59	62	75
3	1	201810	188.1	44	45	58
4	1	201811	192.6	46	47	57

```
trial = monthly_metrics[
    monthly_metrics["STORE_NBR"] == 77
][["YEARMONTH", "Total_Sales"]]

control = monthly_metrics[
    monthly_metrics["STORE_NBR"] == 233
][["YEARMONTH", "Total_Sales"]]
```

```
comparison = trial.merge(
    control,
    on="YEARMONTH",
    suffixes=("_Trial", "_Control")
)

comparison.head()
```

	YEARMONTH	Total_Sales_Trial	Total_Sales_Control
0	201807	296.8	290.7
1	201808	255.5	285.9
2	201809	225.2	228.6
3	201810	204.5	185.7
4	201811	245.3	211.6

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
scale = (
    pre_trial.query("STORE_NBR == 77")["Total_Sales"].sum()
    /
    pre_trial.query("STORE_NBR == 233")["Total_Sales"].sum()
)

comparison["Scaled_Control"] = (
    comparison["Total_Sales_Control"] * scale
)
```

```
comparison["PctDiff"] = (
    comparison["Total_Sales_Trial"]
    - comparison["Scaled_Control"]
) / comparison["Scaled_Control"]
```

```
print(monthly_metrics.columns.tolist())

['STORE_NBR', 'YEARMONTH', 'Total_Sales', 'Customers', 'Transactions', 'Total_Qty']
```

```
trial_store = 77
control_store = 233

# Calculate scaling factor using pre-trial months
scale = (
    pre_trial.loc[pre_trial["STORE_NBR"] == trial_store, "Total_Sales"].sum()
    /
    pre_trial.loc[pre_trial["STORE_NBR"] == control_store, "Total_Sales"].sum()
)

print("Scaling Factor:", scale)
```

Scaling Factor: 1.023617303289553

```
trial_sales = monthly_metrics[
    monthly_metrics["STORE_NBR"] == trial_store
][["YEARMONTH", "Total_Sales"]]

control_sales = monthly_metrics[
    monthly_metrics["STORE_NBR"] == control_store
][["YEARMONTH", "Total_Sales"]]
```

```
control_sales["Scaled_Control"] = control_sales["Total_Sales"] * scale
```

```
comparison = trial_sales.merge(
    control_sales[["YEARMONTH", "Scaled_Control"]],
    on="YEARMONTH"
)

comparison.head()
```

	YEARMONTH	Total_Sales	Scaled_Control	
0	201807	296.8	297.565550	
1	201808	255.5	292.652187	
2	201809	225.2	233.998916	
3	201810	204.5	190.085733	
4	201811	245.3	216.597421	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
comparison["PctDiff"] = (
    comparison["Total_Sales"] - comparison["Scaled_Control"]
) / comparison["Scaled_Control"]

comparison.head()
```

	YEARMONTH	Total_Sales	Scaled_Control	PctDiff	
0	201807	296.8	297.565550	-0.002573	
1	201808	255.5	292.652187	-0.126950	
2	201809	225.2	233.998916	-0.037602	
3	201810	204.5	190.085733	0.075830	
4	201811	245.3	216.597421	0.132516	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
std_dev = comparison.loc[
    comparison["YEARMONTH"] < 201902,
    "PctDiff"
].std()

print("Standard Deviation:", std_dev)
```

Standard Deviation: 0.09958646884078388

comparison["tValue"] = comparison["PctDiff"] / std_dev

comparison

	YEARMONTH	Total_Sales	Scaled_Control	PctDiff	tValue	
0	201807	296.8	297.565550	-0.002573	-0.025834	
1	201808	255.5	292.652187	-0.126950	-1.274771	
2	201809	225.2	233.998916	-0.037602	-0.377585	
3	201810	204.5	190.085733	0.075830	0.761452	
4	201811	245.3	216.597421	0.132516	1.330661	
5	201812	267.3	286.408121	-0.066716	-0.669934	
6	201901	204.4	181.692071	0.124980	1.254993	
7	201902	235.0	249.762622	-0.059107	-0.593520	
8	201903	278.5	203.802205	0.366521	3.680430	
9	201904	263.5	162.345704	0.623080	6.256669	
10	201905	299.3	352.533799	-0.151003	-1.516304	
11	201906	264.7	226.219424	0.170103	1.708092	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
from scipy.stats import t
degrees_of_freedom = 7
critical_value = t.ppf(0.95, degrees_of_freedom)
print("Critical Value:", critical_value)
```

Critical Value: 1.894578605061305

```
trial_period = comparison[
    comparison["YEARMONTH"] >= 201902
]
trial_period[["YEARMONTH", "tValue"]]
```

	YEARMONTH	tValue	
7	201902	-0.593520	
8	201903	3.680430	
9	201904	6.256669	
10	201905	-1.516304	
11	201906	1.708092	

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

plt.plot(
    comparison["YEARMONTH"],
    comparison["Total_Sales"],
    marker="o",
    linewidth=2,
    label="Trial Store"
)

plt.plot(
    comparison["YEARMONTH"],
    comparison["Scaled_Control"],
    marker="o",
    linewidth=2,
    label="Scaled Control Store"
)

# Highlight trial period (Feb-Apr 2019)
plt.axvspan(201902, 201904, color="orange", alpha=0.2, label="Trial Period")

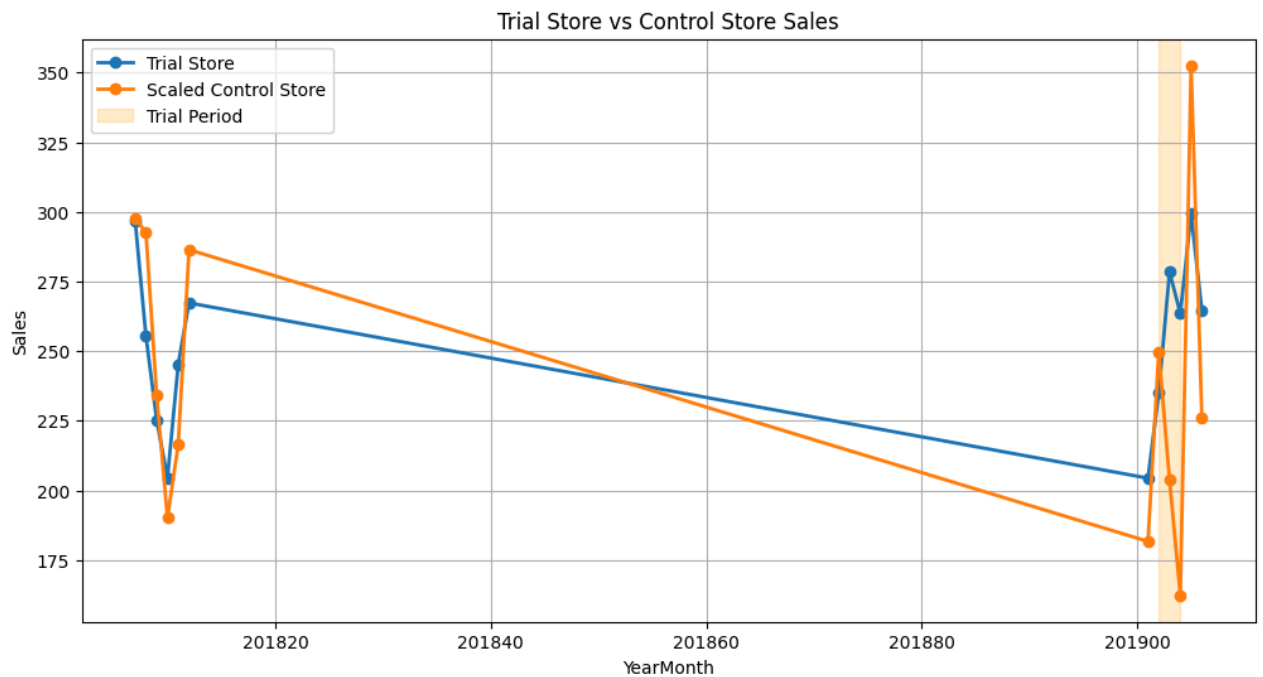
plt.xlabel("YearMonth")
plt.ylabel("Sales")
```



```
plt.title("Trial Store vs Control Store Sales")

plt.legend()
plt.grid(True)

plt.show()
```



```
trial_store = 77
control_store = 233

scale_customers = (
    pre_trial.loc[
        pre_trial["STORE_NBR"] == trial_store,
        "Customers"
    ].sum()
    /
    pre_trial.loc[
        pre_trial["STORE_NBR"] == control_store,
        "Customers"
    ].sum()
)

print(scale_customers)

1.0033557046979866
```

```
trial_customers = monthly_metrics[
    monthly_metrics["STORE_NBR"] == trial_store
][["YEARMONTH", "Customers"]]

control_customers = monthly_metrics[
    monthly_metrics["STORE_NBR"] == control_store
][["YEARMONTH", "Customers"]]
```

```
control_customers["Scaled_Customers"] = (
    control_customers["Customers"] * scale_customers
)
```

```
customer_compare = trial_customers.merge(
    control_customers[
        ["YEARMONTH", "Scaled_Customers"]
    ],
    on="YEARMONTH"
)

customer_compare.head()
```

	YEARMONTH	Customers	Scaled_Customers	
0	201807	51	51.171141	
1	201808	47	48.161074	
2	201809	42	42.140940	
3	201810	37	35.117450	
4	201811	41	40.134228	



Next steps:

[Generate code with customer_compare](#)

[New interactive sheet](#)

```
customer_compare["PctDiff"] = (
    customer_compare["Customers"]
    -
    customer_compare["Scaled_Customers"]
) / customer_compare["Scaled_Customers"]
```

```
std_customer = customer_compare.loc[
    customer_compare["YEARMONTH"] < 201902,
    "PctDiff"
].std()
```

```
std_customer
```

```
0.02743996009077893
```

```
customer_compare["tValue"] = (
    customer_compare["PctDiff"]
    /
    std_customer
)
```

```
customer_compare
```

	YEARMONTH	Customers	Scaled_Customers	PctDiff	tValue	
0	201807	51	51.171141	-0.003344	-0.121884	
1	201808	47	48.161074	-0.024108	-0.878578	
2	201809	42	42.140940	-0.003344	-0.121884	
3	201810	37	35.117450	0.053607	1.953620	
4	201811	41	40.134228	0.021572	0.786149	
5	201812	46	47.157718	-0.024550	-0.894678	
6	201901	35	35.117450	-0.003344	-0.121884	
7	201902	45	45.151007	-0.003344	-0.121884	
8	201903	50	40.134228	0.245819	8.958446	
9	201904	47	30.100671	0.561427	20.460197	
10	201905	55	57.191275	-0.038315	-1.396316	
11	201906	41	41.137584	-0.003344	-0.121884	



Next steps:

[Generate code with customer_compare](#)

[New interactive sheet](#)

```
from scipy.stats import t
```

```
critical = t.ppf(0.95,7)
```

```
critical
```

```
np.float64(1.894578605061305)
```

```
customer_compare[
    customer_compare["YEARMONTH"] >= 201902
][["YEARMONTH", "tValue"]]
```

	YEARMONTH	tValue	
7	201902	-0.121884	
8	201903	8.958446	
9	201904	20.460197	
10	201905	-1.396316	
11	201906	-0.121884	

```
plt.figure(figsize=(12,6))

plt.plot(
    customer_compare["YEARMONTH"],
    customer_compare["Customers"],
    marker="o",
    linewidth=2,
    label="Trial Store"
)

plt.plot(
    customer_compare["YEARMONTH"],
    customer_compare["Scaled_Customers"],
    marker="o",
    linewidth=2,
    label="Scaled Control Store"
)

plt.axvspan(
    201902,
    201904,
    alpha=0.25,
    label="Trial Period"
)

plt.title("Customer Count Comparison")

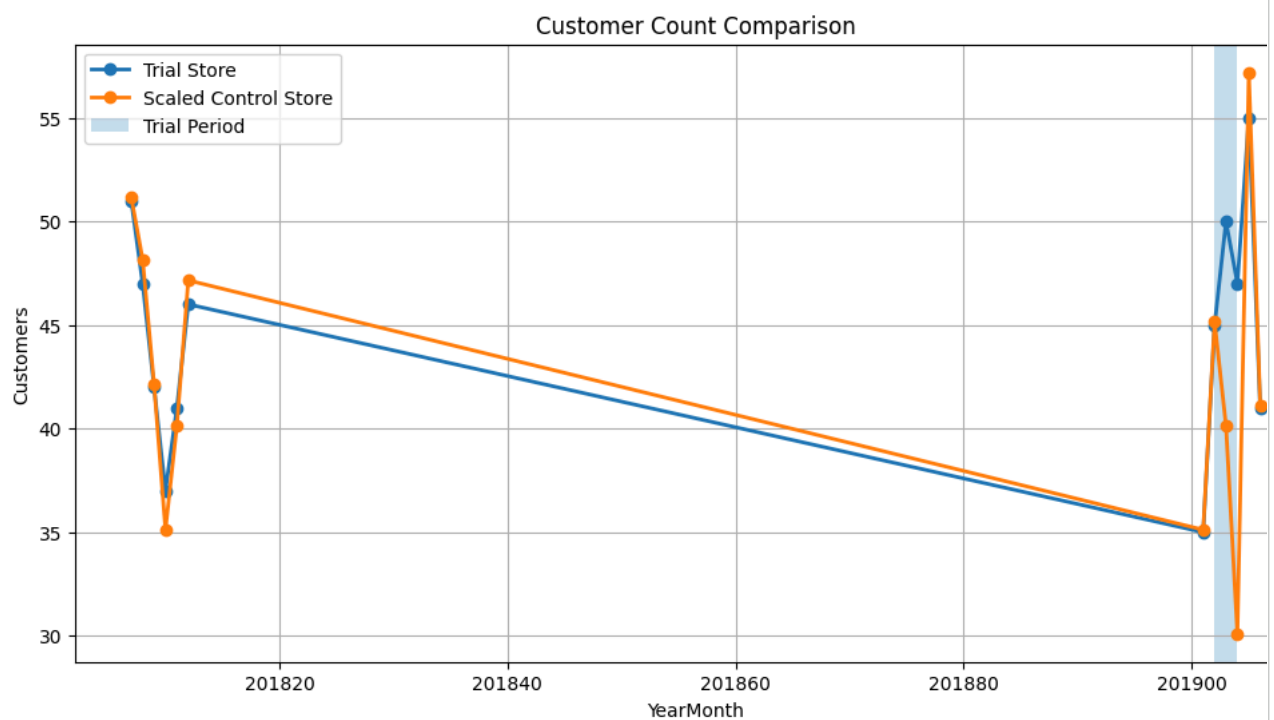
plt.xlabel("YearMonth")

plt.ylabel("Customers")

plt.legend()

plt.grid(True)

plt.show()
```



```
from scipy.stats import t
import matplotlib.pyplot as plt
```

```
def assess_trial(trial_store, control_store, monthly_metrics, pre_trial):
```

```
# -----
# SALES ANALYSIS
# -----

scale = (
    pre_trial.loc[
        pre_trial["STORE_NBR"] == trial_store,
        "Total_Sales"
    ].sum()
    /
    pre_trial.loc[
        pre_trial["STORE_NBR"] == control_store,
        "Total_Sales"
    ].sum()
)

trial_sales = monthly_metrics[
    monthly_metrics["STORE_NBR"] == trial_store
][["YEARMONTH", "Total_Sales"]]

control_sales = monthly_metrics[
    monthly_metrics["STORE_NBR"] == control_store
][["YEARMONTH", "Total_Sales"]]

control_sales["Scaled_Control"] = (
    control_sales["Total_Sales"] * scale
)

comparison = trial_sales.merge(
    control_sales[["YEARMONTH", "Scaled_Control"]],
    on="YEARMONTH"
)

comparison["PctDiff"] = (
    comparison["Total_Sales"]
    -
    comparison["Scaled_Control"]
) / comparison["Scaled_Control"]

std = comparison.loc[
    comparison["YEARMONTH"] < 201902,
    "PctDiff"
].std()

comparison["tValue"] = comparison["PctDiff"] / std

critical = t.ppf(0.95,7)

print("="*60)
print(f"Trial Store : {trial_store}")
print(f"Control Store : {control_store}")
print("="*60)

print("\nSales T-values")

print(
    comparison[
        comparison["YEARMONTH"] >= 201902
    ][["YEARMONTH", "tValue"]]
)

print("\nCritical Value =", critical)

# -----
# SALES GRAPH
# -----

plt.figure(figsize=(12,6))

plt.plot(
    comparison["YEARMONTH"],
    comparison["Total_Sales"],
    marker="o",
    linewidth=2,
    label="Trial"
)

plt.plot(
    comparison["YEARMONTH"],
    comparison["Scaled_Control"],
```

```

        marker="o",
        linewidth=2,
        label="Control1"
    )

plt.axvspan(
    201902,
    201904,
    alpha=0.25,
    label="Trial Period"
)

plt.title(
    f"Sales Comparison : Store {trial_store}"
)

plt.xlabel("YearMonth")

plt.ylabel("Sales")

plt.legend()

plt.grid(True)

plt.show()

# -----
# CUSTOMER ANALYSIS
# -----

cust_scale = (

    pre_trial.loc[
        pre_trial["STORE_NBR"]==trial_store,
        "Customers"
    ].sum()

    /

    pre_trial.loc[
        pre_trial["STORE_NBR"]==control_store,
        "Customers"
    ].sum()

)

trial_customer = monthly_metrics[
    monthly_metrics["STORE_NBR"]==trial_store
][["YEARMONTH", "Customers"]]

control_customer = monthly_metrics[
    monthly_metrics["STORE_NBR"]==control_store
][["YEARMONTH", "Customers"]]

control_customer["Scaled_Customers"] = (

    control_customer["Customers"]

    *

    cust_scale

)

cust_compare = trial_customer.merge(

    control_customer[
        ["YEARMONTH", "Scaled_Customers"]
    ],

    on="YEARMONTH"

)

cust_compare["PctDiff"] = (

    cust_compare["Customers"]

    -

    cust_compare["Scaled_Customers"]

```

```

) / cust_compare["Scaled_Customers"]

std = cust_compare.loc[
    cust_compare["YEARMONTH"]<201902,
    "PctDiff"
].std()

cust_compare["tValue"] = cust_compare["PctDiff"]/std

print("\nCustomer T-values")

print(

    cust_compare[
        cust_compare["YEARMONTH"]>=201902
    ][["YEARMONTH", "tValue"]]

)

plt.figure(figsize=(12,6))

plt.plot(
    cust_compare["YEARMONTH"],
    cust_compare["Customers"],
    marker="o",
    linewidth=2,
    label="Trial"
)

plt.plot(
    cust_compare["YEARMONTH"],
    cust_compare["Scaled_Customers"],
    marker="o",
    linewidth=2,
    label="Control"
)

plt.axvspan(
    201902,
    201904,
    alpha=0.25,
    label="Trial Period"
)

plt.title(
    f"Customers Comparison : Store {trial_store}"
)

plt.xlabel("YearMonth")

plt.ylabel("Customers")

plt.legend()

plt.grid(True)

plt.show()

```

```

assess_trial(
    trial_store=77,
    control_store=233,
    monthly_metrics=monthly_metrics,
    pre_trial=pre_trial
)

```

```

assess_trial(
  trial_store=86,

```

```
control_store=155,  
monthly_metrics=monthly_metrics,  
pre_trial=pre_trial  
)
```


=====

Trial Store : 86

Control Store : 155

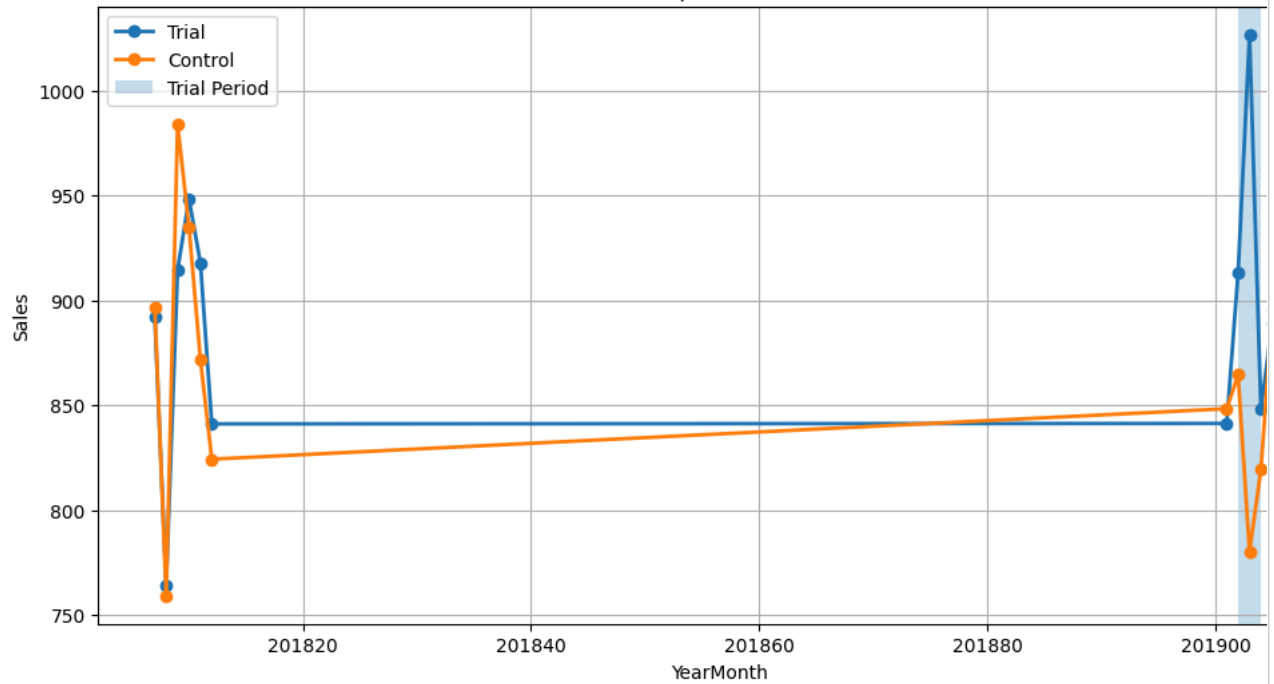
=====

Sales T-values

	YEARMONTH	tValue
7	201902	1.494114
8	201903	8.381769
9	201904	0.935444
10	201905	-0.175613
11	201906	0.206152

Critical Value = 1.894578605061305

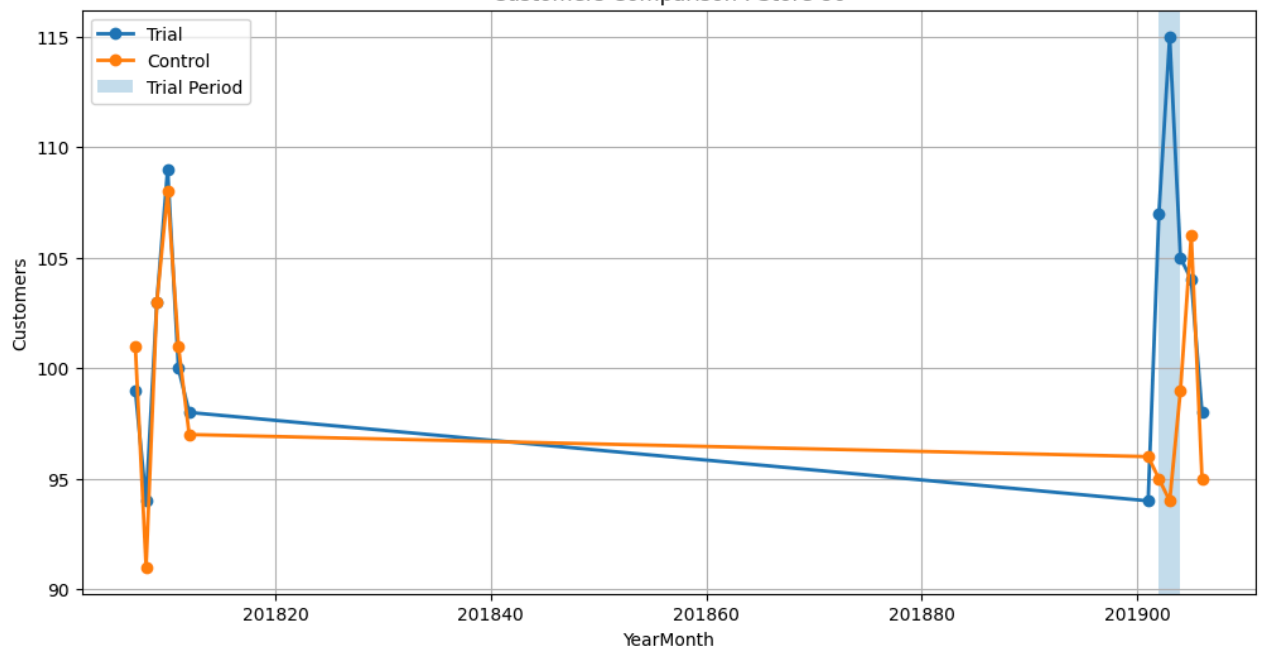
Sales Comparison : Store 86



Customer T-values

	YEARMONTH	tValue
7	201902	6.592964
8	201903	11.660429
9	201904	3.163291
10	201905	-0.984798
11	201906	1.648241

Customers Comparison : Store 86



```
assess_trial(  
  trial_store=88,  
  control_store=237,
```

```

monthly_metrics=monthly_metrics,
pre_trial=pre_trial
)

```

```

=====
Trial Store : 88
Control Store : 237
=====

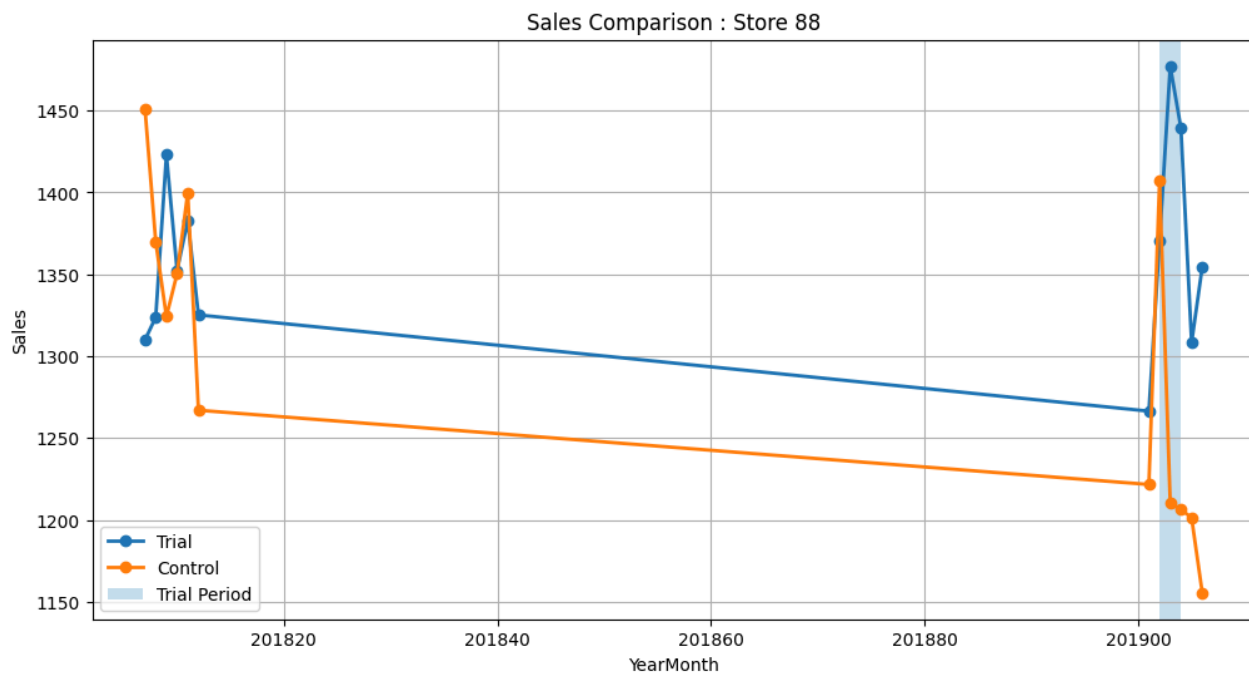
```

```

Sales T-values
YEARMONTH  tValue
7          201902 -0.456726
8          201903  3.855795
9          201904  3.372253
10         201905  1.557169
11         201906  3.011549

```

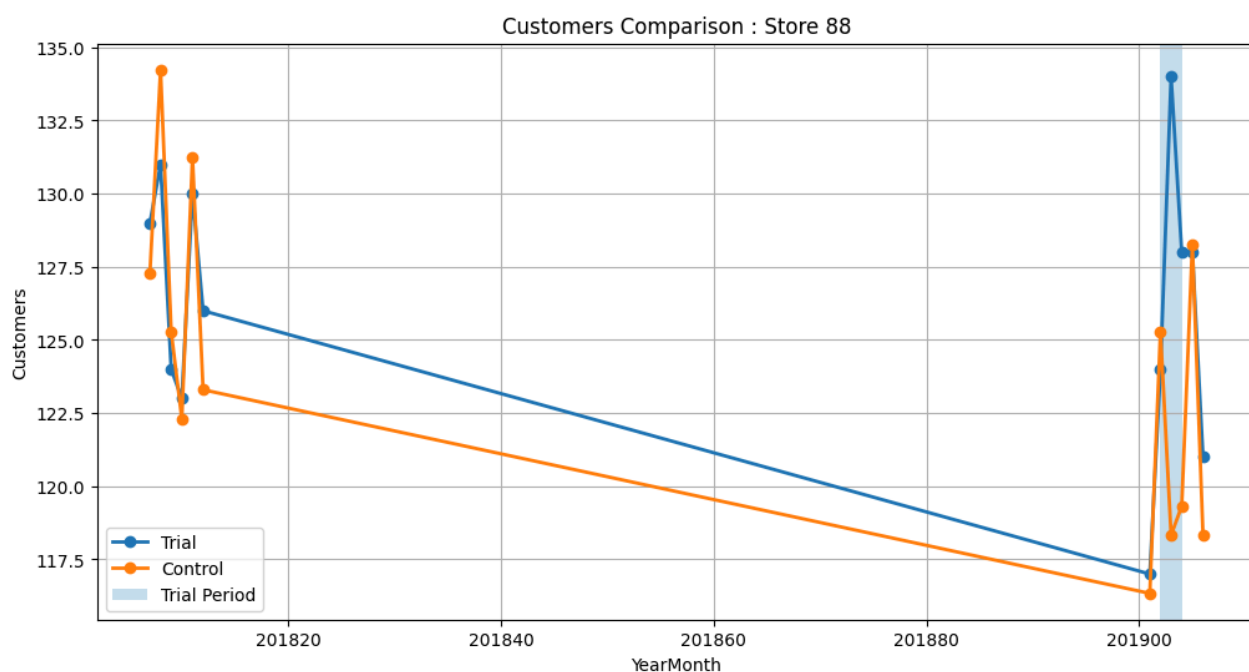
Critical Value = 1.894578605061305



```

Customer T-values
YEARMONTH  tValue
7          201902 -0.649164
8          201903  8.362766
9          201904  4.591985
10         201905 -0.133488
11         201906  1.425951

```



```
summary = pd.DataFrame({
    "Trial Store": [77, 86, 88],
    "Control Store": [233, 155, 237],
    "Sales Uplift": [
        "Significant",
        "Not Significant",
        "Significant"
    ],
    "Customer Increase": [
        "Significant",
        "Significant",
        "Significant"
    ],
    "Recommendation": [
        "Roll Out Layout",
        "Investigate Further",
        "Roll Out Layout"
    ]
})
```

summary

	Trial Store	Control Store	Sales Uplift	Customer Increase	Recommendation	
0	77	233	Significant	Significant	Roll Out Layout	
1	86	155	Not Significant	Significant	Investigate Further	
2	88	237	Significant	Significant	Roll Out Layout	